

BRIDGES: A Framework for Modeling Students' Software Development Processes

Anonymous Author(s)

Abstract

Investigating student processes has become increasingly critical in the era of generative AI, as it enables educators to discern not only the quality of students' outputs but also the nature of their engagement, strategies, and collaboration. Process-oriented analyses can illuminate both individual learning trajectories and the dynamics of team-based work. Building on recent advancements in process mining within software engineering contexts, this study introduces the BRIDGES framework, which categorizes students' software development behaviors into three key domains: project management, productivity, and coding practices. The framework delineates both ideal and non-ideal patterns of student behavior, serving as the foundation for constructing process maps that capture development events. To illustrate the utility of our framework, we examine a subset of events under coding practices that characterize students' communication and code structure maturity using empirical data from senior undergraduate projects. We leverage large language models in classifying the quality of branch names, PR descriptions, and code reviews. Our findings indicate that process maps reveal pronounced distinctions between high- and low-performing teams. Moreover, student-level process maps can explain differential behaviors that appear similar at the team level, thereby enabling educators to identify the underlying causes of performance differences. Across the cohort, we saw natural clusters exhibiting varying levels of engagement, productivity, and code maturity. Ultimately, our goal is to leverage these process maps to provide real-time, data-informed feedback that supports students in their software engineering practices.

CCS Concepts

• Software and its engineering → Software development process management; Programming teams; • Social and professional topics → Computing education;

Keywords

Software development pattern, student behavior, coding practice, team communication, process mining, mining software repositories, software engineering education

ACM Reference Format:

Anonymous Author(s). 2026. BRIDGES: A Framework for Modeling Students' Software Development Processes. In *Proceedings of Make sure to*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

enter the correct conference title from your rights confirmation email (Conference acronym 'XX). ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Educators and learning scientists have long debated the fundamental distinction between learning and performance [19]. Although many empirical studies in educational domains use performance data as a proxy to demonstrate learning improvements, true learning must be independently assessed for knowledge retention and transfer. Rather than indirectly assessing student learning through their work outputs (e.g., test answers, project deliverables, or features implemented) — which are single data points in the learner's journey, researchers have advocated for evaluating the processes that led to the generation of these outputs [2]. Investigations into student processes are especially crucial in the age of generative AI, where simple Git metrics are no longer sufficient for capturing meaningful characteristics of student programs. In contrast, investigating the underlying processes has the potential to shed light on students' learning and team collaborative behavior.

Process mining has gained significant attention over the past decade in the educational data mining and learning analytics community [3, 18]. Using log data, process mining garners insights to uncover behavioral patterns and presents the findings in a visualization called *process map* that better supports instructional interpretation. Process mining has been used for three categories of purposes: process discovery, conformance checking, and process enhancement [18]. Existing literature in educational process mining offers insights into the general patterns in how students solve problems in given domains, differences in the solution approach between multiple student groups, natural groupings of emergent behavior, and efficiency improvements in observed processes.

In the context of agile software development, process mining has been used to enact change in agile business processes, understand user preferences based on their software interactions, and to check for conformance in adhering to agile software development processes [7]. While the literature can also be broadly categorized as process discovery, conformance checking, and enhancement, the focus in this area is on understanding organizational processes and improving agile workflows. Common data sources in these studies include issues (e.g., bugs, tests, features, and backlog items [21]), code repositories (e.g., combining commit activities with other sources such as tickets, bugs, mail archives, and wiki articles [13]), and communication channels (e.g., email actions [4]). In recent work, process mining has also been applied to software engineering education with varying granularity of events to provide insights into learner difficulties. However, the selection of relevant events for constructing process maps remains an open question and is often determined at the discretion of individual researchers.

Analyzing Git activity has significant potential for understanding students' programming behavior. However, determining which activities are relevant to learning is non-trivial. Researchers have proposed incorporating different details in defining relevant events from software repositories. For example, a commit activity from a Git repository may include the code repository as the data source, and the event may be represented based on the document type [15] or using a collection of details such as the commit identifier, time, author, whether line additions or deletions were involved, and the relevant architectural component [13]. As a first step, we propose a framework called BRIDGES to represent different types of software development patterns in student groups by envisioning ideal and non-ideal observable behaviors from students. We demonstrate the utility of the BRIDGES framework by mapping a subset of coding practice behaviors to events in the context of an empirical study involving 22 senior undergraduate team projects. We report on the process maps generated at both student and team levels, and explore the differences between high- and low-performing students. Our specific research questions (RQs) are:

RQ1: How does the BRIDGES framework help to organize and structure existing knowledge on student software development behaviors?

RQ2: What are the challenges and limitations encountered when mapping the framework's behaviors into events in an empirical study, and how can they be addressed?

RQ3: To what extent do the resulting process maps capture the range of software development maturity in students and explain learner difficulties?

We first review the literature on process mining in Section 2, with an emphasis on the applications in software engineering education. We present our framework in Section 3 and the method of the empirical study in Section 4. Our results in Section 5 indicate that process maps reveal pronounced distinctions between high- and low-performing teams. Moreover, student-level process maps can explain differential behaviors that appear similar at the team level, thereby enabling educators to identify the underlying causes of performance differences. Across the cohort, we saw natural clusters exhibiting varying levels of engagement, productivity, and code maturity. Our contributions include: (i) a novel framework that captures the multifaceted nature of software development processes, (ii) a systematic approach that leverages an LLM for mapping behavioral patterns into process events, and (iii) an exploratory study employing process maps and clustering to characterize distinct development tendencies. Ultimately, our goal is to leverage these process maps to provide real-time, data-informed feedback that helps students improve their software engineering practices.

2 Background

In 2019, a systematic mapping study on identifying process mining techniques and applications found 1278 relevant articles in the literature [8]. The authors identified many articles on process discovery algorithms applied to various application domains, including healthcare, technology, logistics, and education. Within the education domain, process mining has been applied in a number of settings, such as massive open online courses, computer-supported collaborative learning, professional training, curriculum mining,

computer-based assessments, and software repositories [3]. The goal of these applications is to discover learner flows and sequential patterns in order to detect learning difficulties.

Around the same time, another systematic mapping study from 2018 scanned the literature for using process mining in agile business and software development, which identified 25 relevant papers mostly published in the last decade [7]. The majority of these papers focused on discovering organizational processes, where some used repository data to analyze conformance to predefined agile methodologies. The authors also found a few papers that modeled user interaction to determine feature usage patterns. Among these, only 5 articles used data from code repositories, and 2 used data from communication channels. The latter studies extracted high-level project activities from emails to identify process delays, but do not offer insights into the social dynamics of the teams. The 5 that used repository data focused on business procedures, software usability, and two early papers [11, 13] that explored software development practices, which we discuss in more detail below.

In an early 2007 publication, Rubin et al. outlined a framework that identified multiple perspectives to software process models, covering control-flow (e.g., task execution order), information (e.g., production of data and documents), organization (e.g., who did which task), and performance (e.g., task time) [15]. Their study demonstrated the framework by constructing control-flow processes with document commits and mapping them to timestamped events (e.g., configuration, code, test). The authors also conducted a conformance analysis on the generated process maps.

Several studies have applied process mining to software engineering education. Early works that applied process mining to software engineering education combined multiple data sources (e.g., team wiki, issue tracking system, version control system) to analyze software development patterns [11, 13, 21]. For example, process maps of workflow activities were created to analyze the lifecycle of the bug fixing process [11, 13] and conformance with the Scrum framework [21]. These earlier studies were instrumental in establishing the foundational direction of the field.

With the widespread adoption of GitHub as a central platform for software development, there is a growing need to examine finer-grained characteristics that provide deeper insights into coding behaviors (e.g., commit and branching strategies) and team interactions (e.g., patterns in code reviews). The focus of the study by Macak et al. is to identify relevant events from Git repositories in a web development course to discover and improve the learner process [10]. Specifically, the authors were interested in understanding the students' commit strategies, testing process, and branching strategies. The study identified events which includes an initial commit and 12 additional coding events categorized as one of positivity (whether a commit had only line additions, only deletions, or both), size (small, medium, big), branches (new or merge branches), and tests (whether a commit only had tests, code and test, or code with no tests). Separate process maps were presented to capture team activities in each category, which were analytically cross-referenced with internal course deadlines to inspect behavioral changes across deadlines. These results are useful in helping students create small commits more consistently and ensuring they follow a test-driven development (TDD) approach. The authors also compared individual process maps to identify which student contributed the most

code changes in a team project. In the age of generative AI, one must be careful not to miscount generated code as contributions.

Process mining has also been applied to small assignments. In one study, source code events pertaining to the modification behavior (e.g., add/remove/rename variables, modifiers, comments, loops, accessibility, data type) of method body, method, field, interface, and class were extracted [16]. The authors used this data to generate detailed process maps and analyzed the development process of specific assignments together with the resulting source code.

One study applied process mining to explore student dynamics using data from an integrated development environment (IDE) and the software repositories for 5 student groups in a second-year programming course [1]. The authors analyzed the process maps for individual students and designated the member with more activity and commits as the “leader” and the others as “followers”. The study found that the students did not make much use of the IDE functions, such as generating getter and setter functions, searching the source code, engaging with the debugger, and exhibiting a low frequency of commits and pulls. No additional information on team interaction and communication was reported.

Pons et al. explored the value of process mining using *contextualized* commits that included situational information about the pedagogical context (such as before or after an intervention) [14]. Although the authors created four types of contextualized commits, these events were not used in any of the process maps produced. Instead, their process maps focused on the order in which assignment questions were completed and how students navigated between questions. The authors recommended further studies to understand the reasons behind the observed execution order and a study on the relationship between that behavior and performance scores.

In an effort to map out the relevant dimensions from studies involving educational process mining, Wambsganss et al. developed a taxonomy to capture event patterns observed from technology-mediated learning environments [18]. The authors developed an initial taxonomy based on qualitative coding of existing literature, refined it using feedback from interviews with experts, and organized the categories based on suggested groupings from cluster analysis. The final taxonomy has dimensions to capture different types of learning purpose, learner context, data source, and analyses used. The review found studies that applied process mining to investigate behaviors in individual and collaborative settings, and applications that explored factual, conceptual, procedural, and metacognitive outcomes. Given this taxonomy, we situate our work in the collaborative learning context involving Git data to examine software development behaviors for the purposes of discovering and improving students' procedural processes.

3 The BRIDGES Framework

We propose a framework to identify relevant software development events for understanding Behavioral Representation and Interaction Dynamics in Group Environments for Students (BRIDGES). The BRIDGES framework is motivated by a broad set of GitHub development activities (e.g., [12]) and models the behavioral connections that form within student teams, linking individual actions to collective dynamics and learning outcomes. It serves as a behavioral analytics framework that bridges individual and collaborative activities

to better understand student engagement and teamwork in software development. The framework has three different knowledge domains: **project management**, **productivity**, and **coding practices**. Each domain is evaluated along a spectrum of ideal versus non-ideal behaviors from the perspective of software engineering. Ideal behaviors are consistent with professional standards, maintainability, constructive communication, and teamwork, whereas non-ideal behaviors tend to introduce inconsistencies, confusion, technical debt, unbalanced work, or poor team coordination.

3.1 Project Management

Project management addresses how teams organize and track their work on GitHub. Teams should ideally create issues with clear descriptions and necessary details, size issues appropriately, assign issues regularly, resolve issues in a timely manner, update task status, maintain a backlog of tasks, ensure balanced workload distributions, use the project board consistently, and communicate regularly with team members. In contrast, dysfunctional teams may exhibit non-ideal behaviors, such as poor issue descriptions, inaccurate issue sizing, untracked work, taking too long to resolve issues, outdated information, missed deadlines, inconsistent use of the project board, and a lack of communication with each other. Project management improves collaboration quality by providing structure, enhancing communication, and optimizing workflows.

3.2 Productivity

Productivity captures the development and sustainability of contributions from the team members. Ideal behaviors show steady, incremental progress by all the team members, where the time spent tackling the work is proportional to the size of the issues. High productivity should also reflect a consistently high amount of task completions and short bug lifecycles, where high-priority tasks are focused on first. On the other hand, non-ideal patterns include long gaps followed by rushed commits to meet deadlines or over-reliance on a single contributor. Low productivity also includes irregular commit patterns, long bug lifecycles, delayed reviews, and tasks that block others from their work. Productivity is important for collaboration quality because it leads to increased engagement and strengthens relationships and trust in the team.

3.3 Coding Practices

Coding practices pertain to aspects of code quality, readability, maintainability, collaboration processes, and compliance with pre-established standards. Since most of the software engineering education literature has focused on this area, we decomposed this knowledge domain further into 5 different dimensions: *branching*, *pull requests*, *code structure*, *balanced work*, *communication*. Each dimension contains a list of criteria with associated ideal and non-ideal tendencies shown in Table 1. These tendencies are defined as observations that can be measured from repository data using the GitHub API. We note that certain behaviors — such as the use of meaningful branch names, informative PR descriptions, and constructive code reviews — require qualitative coding techniques to be accurately identified. In these cases, we leverage a large language model (LLM) for automated labeling support, with the specific model configurations detailed in our behavior-to-event mapping

discussion. Coding practices are important for collaboration because they create a consistent, readable codebase that facilitates knowledge sharing and faster development. Our empirical study will draw from a subset of these dimensions and behaviors.

4 Method

4.1 Course Context

In professional software development settings, teams of programmers work together to build and test software. These teams are commonly self-managed following an agile process where tasks are defined, prioritized, and assigned via a project board. We gathered data from a final-year undergraduate software engineering project course that mimics this industry process while students focused on designing and building a project that ran for two academic semesters over eight months. Students worked on one of four possible open-source project options for the entire course. Aside from planning and design activities, students concentrated on programming tasks for approximately two-thirds of the course. All the teams used GitHub as the common development platform.

4.2 Participants

In total, 105 students enrolled in the 2023 course offering at a North American university. Among them, 89 self-identified as men, 15 as women, and 1 as non-binary. Since the student population did not have a concentrated visible minority group, we asked if students were of European descent. As a result, 42 students self-identified as having European descent and 63 did not. Students were assigned to teams of 4 or 5 based on their friendship, goal-orientation, and elicited skills, forming a total of 22 teams.

4.3 Procedure

The teams chose one of four options for their project. Students were asked to follow a weekly code contribution and review process, consisting of regular coding, testing, and code reviews for their project. An additional constraint placed on the software development process is that instructors require students to perform at least two code reviews before a pull request (PR) can be merged. However, anyone on the team can perform a review, and an explicit reviewer assignment was not required. Thus, students were aware that their code reviews were being monitored by the teaching staff.

4.4 Data Collection

We collected data directly from each team's GitHub repository, allowing us to perform a thorough analysis of the codebase. We developed a custom Python-based extraction framework utilizing GitHub's REST API v3 to transform repository data into structured datasets suitable for analysis and process mining. Our Python extraction framework takes advantage of GitHub's REST API to access collaboration metadata by utilizing personal access tokens to increase our rate limit from 60 to 5,000 requests per hour. Each repository had data collected from 8 API endpoints, each with a specific use case for PRs, commit history, file changes, review comments, and branches. Our replication package¹ contains all the prompts and code files used to extract and analyze repositories.

¹Available at: <https://zenodo.org/records/17433312>

4.5 Data Pre-processing

Several pre-processing steps were performed to ensure accurate results before exporting the data to CSV files for analysis. GitHub initialization was filtered to distinguish code changes and documentation updates. PR titles containing any mention of "logs" and files in the log/ path were omitted from code change metrics as these represent students' weekly updates that did not impact the project and were instead used for grading purposes.

The filtered data was converted into 3 CSV files per repository by utilizing Python's pandas library. The first CSV contained pull request metadata, including merge status, reviewer information, and the branch details. Commit-level data, including file-level changes, was exported to the second CSV. The third CSV contained review comments with timestamp information. Utilizing this structure ensured a consistent format to analyze code changes, branching techniques, comments, and discussions across the 22 repositories.

4.6 Event Mapping

To demonstrate the utility of our framework, we model the behaviors in the *coding practices* knowledge domain from Table 1. We map the extracted data to timestamped events that correspond to each of the listed behaviors. Since some of the behaviors involved understanding the quality of the naming conventions and documentation, we used the llama3.2:3b LLM to assist with the classification task set to a temperature of 0.2 for consistent and deterministic classifications. We chose this model as its 3-billion parameter size provides sufficient semantic understanding for our text classifications and relatively lightweight allowing it to run easily on most hardware. The LLM classifications were run locally to ensure reproducibility; complete timestamped classification prompts and reasoning outputs were saved in CSV files for validation. The resulting timestamped events serve as input to the process mining tool. We explain the event mapping of the behaviors below.

4.6.1 Branching. To identify the branching strategy, we analyzed the head branch (head_branch) of each pull request to determine if branch reuse was performed. If multiple PRs were associated with the same branch name, it indicated that multiple features were implemented within a single branch and was deemed as non-ideal. Branches used only once were considered ideal.

The meaningfulness of branch names was evaluated for clarity and purpose in relation to the associated PR title and description. Examples of branch names such as fix/ui-bug and feature/login were considered meaningful. Generic or unrelated branch names such as test or fix were deemed to be too obscure and non-ideal. We used the LLM classified branch names as either *meaningful*, indicating that there is a clear relationship between the PR and the branch name, or *random*, indicating when no alignment was found. When the LLM was unsure, it returned the label *unmeaningful*, which we merged with the random data.

4.6.2 PR Practices. Several criteria were used to understand how teams managed code collaboration, review quality, and merge control. Understanding if PRs have clear descriptions was used to assess how effectively developers documented their code. Using the body field from each PR, descriptions containing fewer than 10 words were considered unclear and too vague for the reviewer to grasp

Table 1: Coding Practices Rubric. Criteria denoted with * have been implemented in our empirical study.

Dimension	Criteria	Ideal Behavior	Non-Ideal Behavior
Branching	*Branching strategy	One branch per feature	Multiple features in a single branch
	*Branch naming conventions	Descriptive and meaningful names (e.g., feat, login)	Generic names (e.g., developer's own name, bug, test, new)
PR Practices	*Clear PR descriptions	Purpose, changes, test steps included	Empty or minimal PR description
	Provide detailed context	Purpose and complexity in commit messages	Vague commit messages ("update" or "fix")
	*Constructive code reviews	Approves PRs only if clean with constructive comments	Non-meaningful approvals ("lgtm") or failing tests
	*Involvement in approval process	Multiple developers review and approve the PR before it is merged excluding the author	One person controls merges or self-merges PR without multiple reviews
	Addressed change requested	Change requests are addressed or allocated to a new PR before merge	No new commits or new PR resulting from change request
	*Changes requested when necessary	Rejecting a PR when problems arise	Skips review and approves PR
Code Structure	*Feature code size	Small code commits	Large code commits
	*Refactor code size	Incremental refactor commits	Large refactors
	*Keep repository up-to-date	Pulling latest changes	Ignoring merge conflicts
	Modular contributions	Work distributed across components	Changes concentrated in a single file (<i>god</i> files)
	*Code integration	All changes are linked to a PR	Changes are pushed directly to the main branch without association to a PR (<i>orphan</i> commits)
Styling	Labeling conventions	Consistent, descriptive naming ("camelCase" versus "snake_case")	Inconsistent, confusing, or unrelated to code
	Inline comments	Functions and complex logic are documented with comments	No comments, poor/unclear comments, or excessive comments
	Adherence to style guides	Consistent format following style guides	Ignores conventions or inconsistent formatting
Testing	Tests written for features	New features contain test cases	No new tests for new features
	Tests pass before merging	Unit, integration, and end-to-end tests pass in continuous integration pipeline before merge	Tests do not pass before merge
	Proper test coverage	Positive and negative cases written and passed	Use only one test case, low test coverage
	Thorough testing process	PR merged only after continuous integration pipeline passes and reviewers approve	PRs merged without testing or review
Balanced Work	Equal contributions	Multiple contributors active across repository	One member dominates activity
	*Proactive code reviews	Different members serve as first and second reviewers at different times.	Certain individuals consistently serve as the first or the second reviewer.
Communication	Review discussion depth	Many comments from multiple contributors	No or minimal communication in PR comments
	Documentation	Features includes documentation	Features merged with no documentation

thoroughly. Otherwise, the descriptions are considered clear. Clear PR descriptions are crucial for providing more context to the reviewer, enabling more thorough reviews.

To determine if commit messages provided context, the messages could be processed for mentions of what the change involves (e.g., new code addition, a change, a bug fix, or a refactor) and any accompanying information, such as an associated issue, a rationale for change, or the resulting improvement. Commit messages that are too short or too vague are considered to be unclear (e.g., "fixed bug", "renamed variables", "fixed spelling", "added comments") and part of non-ideal behavior.

Code review quality was defined based on whether the PR review comments provided constructive feedback. To classify whether a code review comment was constructive, LLM was used to check if the comment addresses defects, suggests improvements, or implies styling issues.

The approval process for PRs was also analyzed using the review data. For each PR, we checked the number of distinct reviewers involved before a merge was made to the main branch. PRs with two or more approvals made by other team members were considered ideal as they showcased peer evaluation of code to increase knowledge sharing among teammates. Self-merged PRs or PRs merged with fewer than two reviews were deemed unideal.

Reviewers' tendency to request changes to PRs when necessary was checked by examining each PRs review state history. Using the `changes_requested` status before approval indicated that there were issues with the proposed PR, and the reviewer did not simply approve to have the work merged by a deadline. An ideal team behavior would showcase this metric used within reason throughout the week, as the presence of change requests implies thorough reviews and a desire to maintain good standards. PRs merged without any changes requested do not necessarily entail non-ideal behavior;

however, we do expect some change requests in all the projects based on the current level of programming quality observed.

4.6.3 Code Structure. Understanding how each team maintained and structured the code in their repository is crucial for helping students engage in professional software development practices. The first criterion pertains to the size of code commits, ensuring that their feature code is small in size and easy to review. In general, we analyze each commit to detect refactoring activity. If refactoring activity is detected, the associated code is removed and the rest of the commit is considered as feature code. For the remainder of the commit, we consider the total lines modified (lines added plus lines deleted) as the code size. If the code size is less than 50 lines, then we classify that as a small code commit, which would be an ideal behavior, and otherwise the code size is labeled as a large code commit, which would be a non-ideal behavior.

Each commit was classified as a refactor if it contained lines deleted in an already existing file. For each commit that has detected refactoring activity, the associated code is analyzed for its size. If the refactoring activity's code size is less than 50 lines, it is considered a small refactor, potentially involving incremental changes, which would be ideal. Otherwise, we consider the activity to be a large refactor, which would be non-ideal.

The third criterion assesses how teams handled merging. Using the `behind_by` field in the PR data, a PR exhibits ideal merging patterns if the branch is up-to-date with the base branch before merging (`behind_by` = 0). On the contrary, PRs exhibit non-ideal merging patterns if the branch was behind the base branch, which can lead to more merge conflicts and integration issues.

Although we did not implement this in our study, another criterion relates to modular contributions that could be measured by analyzing the file paths changed in each PR. By understanding the percentage of changes to certain files within each repository, we can determine if modular work was being done or if certain files were becoming "god files" (one file that contains a lot of code). For example, PRs with more than 70% of the total line changes within a single file could be considered as non-ideal due to the over-reliance on one file and a monolithic structure. On the other hand, PRs that involve changes across multiple files showcase a modular design and use more than one component.

The last criterion checked if all commits are integrated. Teams that created a PR for their code exhibited ideal code integration behavior, as it ensured that contributions were reviewed before integration. Direct commits to the main branch were considered as non-ideal behavior as they bypass the review process designed to prevent unwanted or broken code in the main branch.

4.6.4 Styling and Testing. We did not include the behaviors from these dimensions in our study. For styling, consistent formatting and conventions could be checked if linters are used or if style guides are available. The appropriate amount of comments can also be estimated based on the complexity of the code. For testing, one could check the tests written (e.g., code in a `test/` directory) and the results of test reports relative to feature commits to gauge whether high-quality tests were written and passing before merging.

4.6.5 Balanced Work. Balanced work focused on developer contributions across the project. This dimension is crucial in helping to

evaluate students fairly in software engineering education. Equal contributions can be assessed by aggregating the number of commits, merged pull requests, completed reviews, and comments to capture event-level patterns of individual participation. Teams in which members demonstrate comparable levels of contribution across all activity metrics are considered to exhibit ideal collaborative behavior, as this balance reflects equitable opportunities for learning and skill development. Teams with disproportionate participation patterns are considered less effective. Such imbalances often arise when certain members dominate the project, or when individuals remain confined to their areas of comfort, thereby limiting both personal growth and the team's overall functioning.

In our study context where we required each PR to have two reviewers, we are interested in knowing whether code reviews were done proactively. Our experience indicates that some students consistently serve as the second reviewer by repeating a subset of the review content from the first review. Thus, another aspect of balanced work includes understanding the review dynamics within each team. Comments were sorted by the PR timestamps and then ranked with a comment placement order. The first review was labeled as first, the second review as second, and additional reviews were labeled as additional (although this was rare).

4.6.6 Communication. We developed two criteria to assess team communication based on code artifacts. The first emphasizes the PR discussion depth by analyzing the number of review comments from different reviewers per PR. This metric assesses team engagement and shared knowledge. This was not implemented in our study.

Project documentation was measured to understand if the code being merged had associated documentation to explain changes and functions to the code base beyond inline comments. Commits that included changes to a markdown file in the repository or a file in the `doc/` path are considered to be documentation updates. Although not every PR should have feature documentation, teams should strive to maintain project specifications that accurately reflect the current state of implementation. Conversely, teams that prioritize feature development while neglecting regular documentation updates risk creating inconsistencies between their project descriptions and the implemented functionality.

4.7 Process Mining and Clustering

Timestamped events are input into a process mining tool to generate a visual representation of the activities called *process map*. A process map is a state transition diagram where the nodes represent states and the directed arrows represent transitions from a source state to a destination state. Process maps typically have a start and end state, designating the beginning and end of a process being modeled. Given a set of events, we used Python packages `pandas`, `networkx`, and `graphviz` to generate the transition matrix, build the directed state diagram structure, and visualize the process map. Process maps are helpful in determining the common sequences of activities that occur in a team or for an individual, as well as highlighting unusual activities that warrant further attention.

Our timestamped CSV data enables the calculation of event frequencies at the PR level and at the commit level. To convert this raw data into a standard unit of measurement, we use z-scores to describe the data's relative position in a distribution [5]. In a

hypothesis test of significance, common cutoff values used are $\alpha = 0.10$ for detecting marginal significance and $\alpha = 0.05$ for statistical significance, with corresponding z-scores ± 1.645 and ± 1.96 respectively. To determine the significance of each sequence transition, we calculated z-scores and applied the cutoff value of $\alpha = 0.10$ to keep more transitions for comparison context.

We are interested in exploring the general software development patterns of behavior exhibited by the study cohort. To explore commonalities across teams, we applied k-means clustering to the significant activity sequences [17]. Using only significant activity sequences allows us to focus on patterns that stand out for each group, rather than analyzing all the activities collectively.

5 Results

The repository data of 22 teams were collected into a dataset with the average project and average PR statistics, where appropriate, shown in Table 2. We note the high counts reported for the number of lines of code that is likely due to generated code (e.g., boilerplate or AI) and accidental commits of resource files.

Table 2: Project and PR characteristics (mean, standard deviation in brackets).

Characteristic	Project Statistics	PR Statistics
Num. branches	97.1 (± 148.2)	—
Num. PRs	147.7 (± 145.3)	—
Num. commits	3,365.5 ($\pm 2,876.0$)	27.53 (± 23.3)
Num. files	1,135.2 ($\pm 1,029.5$)	10.89 (± 10.5)
Num. code lines	3,280,621.9 ($\pm 6,184,373.4$)	16,685.2 ($\pm 15,034.6$)
Num. reviews	314.5 (± 437.9)	1.9 (± 1.2)
Num. comments	189.5 (± 187.34)	1.3 (± 0.6)
Num. merges	111.6 (± 71.13)	0.8 (± 0.1)

Given the implemented events, we separated them into two sets of activities and visualize their process maps in the discussion below.

5.1 PR Practices

This section discusses PR practices. We include the proactiveness of code reviews from the balanced work dimension here because it deals with the order of reviews conducted which is captured as part of the PR process. The PR practices graph involves states pertaining to the PR descriptions being either clear or unclear. Review comments can be empty with an approval or involving textual comments that can be (non-)constructive as a first, second, or additional review. A PR can be merged by the PR author, merged after a proper review process by teammates, or left unmerged. We define the following groups of states:

- PR description states: clear or unclear
- Review states: empty comment, first, second, or additional and jointly constructive or non-constructive
- Merge states: no merge, self-merge, reviewed merge

Figure 1 shows the PR practices process map for a high-performing team. We see that the team has a general tendency to engage in clear and constructive communication, and in some cases even engage in more than two reviews beyond the class requirement. At the same time, there are some occurrences of poor practices, such as merging

one's own work without review, writing non-constructive reviews or leaving empty comments, and simply approving the work.

A closer look at the individual student process maps show that the team has a mix of high- and low-performing students. In Figures 2 and 3, we see the activity for a stronger and weaker student in the team respectively. The stronger student emphasizes clear communication and engages in some constructive reviews, while the weaker student barely contributes to the project and only conducts non-constructive reviews.

We considered all the transitions and identified the significant transitions in the data by calculating the z-scores of the transitions. The resulting data was used as input to k-means clustering. This analysis produced 6 clusters based on the silhouette scores and interpretability. The resulting clusters are shown in Figure 6, where only significant states and transitions are illustrated. Cluster 1 is unique in that there is a split between the clarity of PR descriptions but subsequently transitioning to a series of non-constructive or empty comments in order to adhere to the review process. Cluster 2 has mostly unclear PRs, followed by non-constructive comments in their review process. Cluster 3 stands out in that it has many self-merged activities as well as additional reviews that are non-constructive. Cluster 4 is similar to Cluster 3, but with much lower frequencies. Cluster 5 is unique in its many unclear PRs and self-merging activities. Lastly, Cluster 6 is similar to Cluster 2, but with lower frequencies.

5.2 Branching and Code Structure Practices

This section discusses activities pertaining to branching and code structure together to gain insights into the students' general approach to development. Branches may contain one or multiple features and their names can be meaningful or random. Code commits can be large or small feature development, and may involve refactoring small or large changes. The sequence of activities can also make the repository outdated if the codebase is not regularly maintained as up-to-date. Some PR work that is deemed extraneous ends up being closed or remains open but unmerged. The same merge states from PR practices are used here.

- Number of features per branch states: one, multiple
- Branch name states: meaningful, random
- Feature size states: small, large
- Refactor size states: small, large
- Repository status states: up-to-date, outdated
- Extraneous PR states: closed, still open
- Merge states: no merge, self-merge, reviewed merge

Figure 5 shows the process map for a high-performing team. This was a highly productive team, that generally had good coding practice such as using one feature per branch, having meaningful branch names, keeping the repository up-to-date regularly, making small commits and refactors, and engaging in reviews before merge. On the other hand, the process map for low-performing teams typically have less optimal branching behaviors, with various PRs that they closed and re-opened, or PRs that remain open and unmerged.

Following the same methodology to get clustered behaviors for branching and code structure, we computed the z-scores of the transitions and ran k-means clustering. This analysis produced 2 clusters, showcasing the main differentiating factor between the

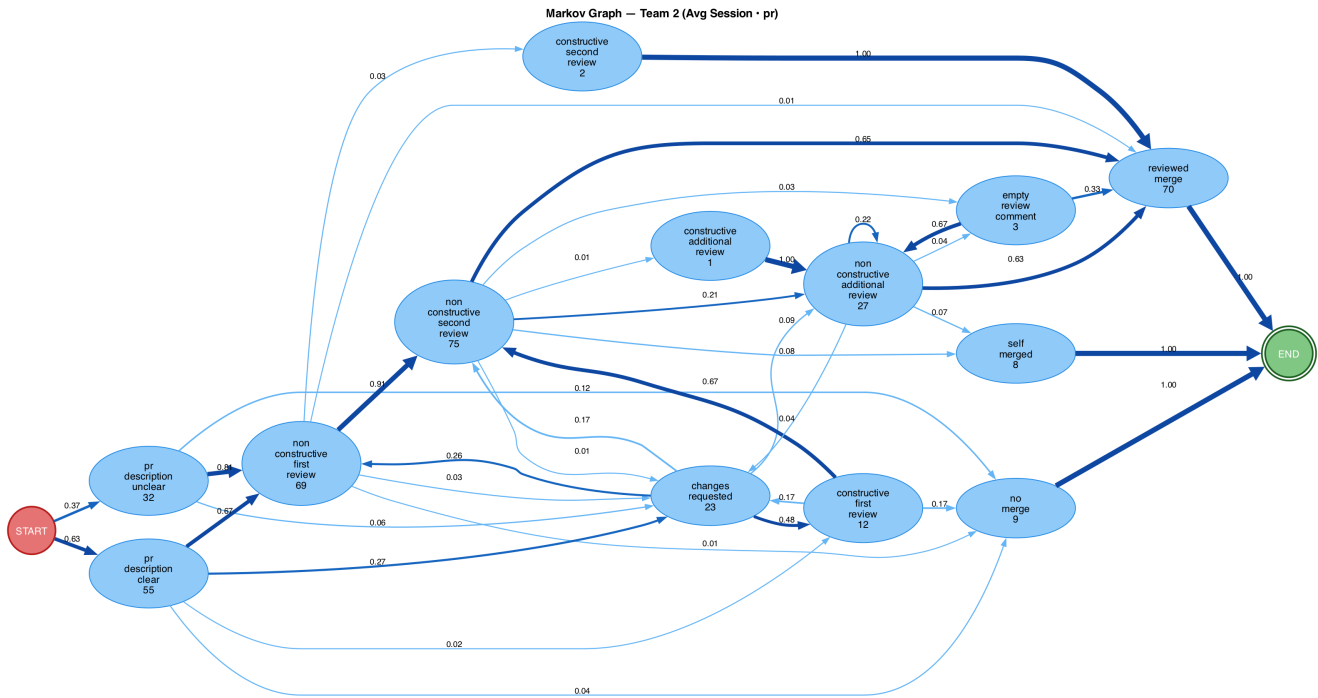


Figure 1: The process map illustrating PR practices for a high-performing team.

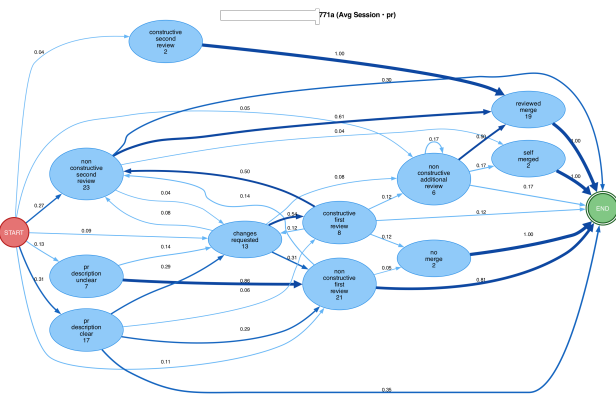


Figure 2: The process map illustrating PR practices for a high-performing student.

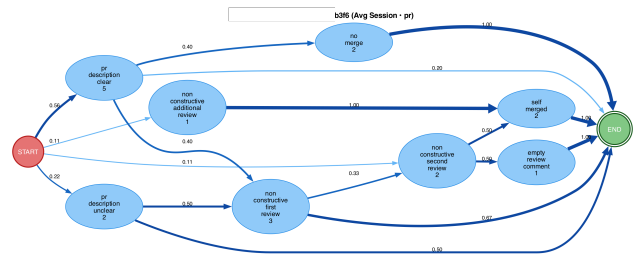


Figure 3: The process map illustrating PR practices for a low-performing student.

6 Discussion and Conclusion

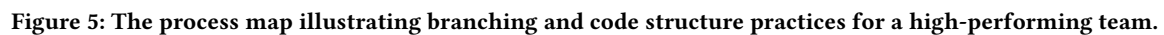
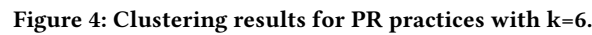
We developed a framework to model ideal and non-ideal student behaviors in software development and demonstrated its applicability through an empirical study that implemented a subset of these behaviors to generate corresponding process maps using repository data from an undergraduate course. The rest of this section discusses the limitations, research questions, and future work.

6.1 Threats to Validity

Our research looks at data from a single cohort that experienced the same curriculum, teaching approaches, assessment structure, and project options. Thus, the course policies employed and the diversity of the students' skills and attitudes may not generalize.

Several of our metrics rely on chosen thresholds chosen by two of the authors based on their teaching experience and understanding

two being the amount of refactoring code committed. Cluster 1 had many more refactors, and significantly more so for small ones than for large ones. We see that even after completing large refactors, about half the time the transition leads to a small refactor. Cluster 2 had fewer refactors in general, although the emphasis was also on small over large refactors.



tools). A related threat to validity concerns the use of LLM for event classification. While LLM-based labeling enables scalable analysis, its reasoning may introduce bias toward certain trends and may not always align with human expert judgments.

9

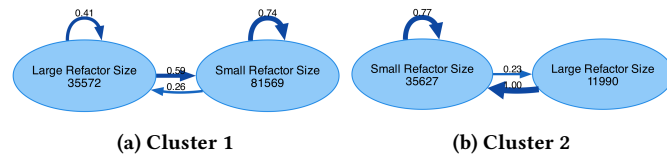


Figure 6: Clustering results for branching and code structure practices with $k=2$.

6.2 RQ1: Framework for Structuring Student Software Development Behaviors

Software development is an inherently multidimensional activity that requires effective planning, teamwork, problem-solving, and adherence to coding practices. The BRIDGES framework offers a comprehensive perspective on students' software development processes by categorizing ideal and non-ideal behaviors across three key knowledge domains: project management, productivity, and coding practices. Together, these domains encompass the range of competencies essential for successful team-based software projects.

In this work, we focus specifically on the coding practices domain, which we further decompose into seven dimensions, each defined by multiple criteria and illustrative examples of ideal and non-ideal behaviors. While the ideal behaviors are grounded in established professional standards, the non-ideal behaviors are informed by prior observations of student work. Categorizing behaviors across these domains enables educators and researchers to examine not only what students do, but also how different forms of knowledge interact and evolve during the development process.

The BRIDGES framework thus provides a holistic theoretical and analytical lens for investigating how students acquire, apply, and integrate diverse forms of knowledge in authentic software engineering contexts.

6.3 RQ2: Lessons from Event Mapping

A key challenge we faced was mapping the ambiguous behaviors (e.g., generic branch names, minimal PR descriptions, vague commit messages, large code commits), which required subjective interpretation of the available information. Manual labeling or rule-based classification can introduce subjectivity, while automated methods may overgeneralize. This, in turn, poses risks for construct validity because the measures may not truly reflect the intended behavior.

GitHub imposes rate limits on the number of requests that can be made within a given timeframe, which introduced delays when retrieving data from larger projects. A key technical challenge involved integrating data from multiple GitHub endpoints while maintaining consistency and completeness across sources. Initially, data collection was performed using PR-based queries. However, this approach excluded orphan commits. Consequently, the data retrieval strategy was revised to ensure comprehensive coverage of all relevant development activities.

Finally, our analysis integrated events across multiple dimensions and criteria into a single process map. This required additional processing steps to ensure that event states did not overlap and that activities were not double-counted. For instance, a single code

review was represented by one unified state, even when it exhibited multiple characteristics (its order and content constructiveness).

6.4 RQ3: Modeling Software Development Maturity and Learner Difficulties

In the era of generative AI, it is increasingly important to support students in developing robust software development skills that foster critical thinking, emphasize system design, and add value to the software engineering process. Process maps of student activity hold significant potential for enhancing learning by enabling feedback that reflects not only the quality of students' project outputs but also the processes through which those outputs are produced.

Our framework provides a structured foundation for modeling software development behavior, identifying a range of mastery across the different skill dimensions. At the individual level, process maps can reveal variations in student competencies, distinguishing those who have mastered standard practices from those exhibiting atypical or inefficient patterns. Such insights allow instructors to identify potential learning difficulties and conduct timely interventions when necessary. This approach facilitates targeted, personalized feedback that is responsive to each student's development trajectory. Overall, applying process mining to students' software development practices enables actionable insights that inform the design of learning activities and assessments. It also supports evidence-based evaluation of teaching interventions and educational technologies by connecting behavioral data with conceptual and social aspects of learning.

6.5 Future Work

Our framework is extensible and can incorporate additional observations about code structure [9], test process [10], code review smells [6], and social relationships [20]. There are opportunities to combine existing software analyses with our framework to provide further insights into student and team development processes from an educational software engineering perspective. For example, cyclomatic complexity, number of function calls, and code churn could represent commit complexity. Implementation of static analysis tools could detect code smells and performance issues which could be used in partner with our event mapping approach. Utilizing behavioral patterns explained within our framework in partner with different code quality metrics could provide more insights into how students develop software.

Future work could include experimenting with the sensitivity of the different thresholds in the event mappings to assess the impact on the interpretation of behaviors. Although many students in this cohort did not use generative AI, future research that distinguishes generated code commits would provide deeper insights into student development behaviors. Additional studies include exploring the relationship between the coding practice behaviors and student performance, as well as investigating the effectiveness of the process maps in delivering real-time feedback to students, potentially identifying critical moments where intervention or support can be most effective.

References

- [1] P. Ardimento, M.L. Bernardi, and M. Cimitile. 2021. Using Process Mining to Understand Students' and Teams' Dynamics. In *Proceedings of the International Workshop on Higher Education Learning Methodologies and Technologies Online (HELMeTO)*. 63–73.
- [2] J. Biggs, D. Kember, and D.Y.P. Leung. 2001. The revised two-factor Study Process Questionnaire: R-SPQ-2F. *British Journal of Educational Psychology* 71 (2001), 133–149.
- [3] A. Bogarín, R. Cerezo, and C. Romero. 2018. A survey on educational process mining. *WIREs Data Mining and Knowledge Discovery* 8, 1 (2018), 1–17.
- [4] C. Di Ciccio and M. Mecella. 2013. Mining Artful Processes from Knowledge Workers' Emails. *IEEE Internet Computing* 17, 5 (2013), 10–20.
- [5] F. Coolidge. 2021. standard scores, the z distribution, and hypothesis testing. In *Statistics: A Gentle Introduction*. SAGE Publications, Inc., 108–142.
- [6] E. Doğan and E. Tüzün. 2022. Towards a taxonomy of code review smells. *Information and Software Technology* 142, 106737 (2022), 1–24.
- [7] S. Erdem, O. Demirörs, and F. Rabhi. 2018. Systematic Mapping Study on Process Mining in Agile Software Development. In *Proceedings of the International Conference on Software Process Improvement and Capability Determination (SPICE)*.
- [8] C.d.S. Garcia, A. Meincheim, E.R. Faria Junior, M.R. Dallagassa, D.M.V. Sato, D.R. Carvalho, E.A.P. Santos, and E.E. Scalabrin. 2019. Process mining techniques and applications – A systematic mapping study. *Expert Systems with Applications* 133 (2019), 260–295.
- [9] H. Keuning, B. Heeren, and J. Jeuring. 2017. Code Quality Issues in Student Programs. In *Proceedings of the ACM Conference on Innovation and Technology in Computer Science Education*. 110–115.
- [10] M. Macak, D. Kruzelo, S. Chren, and B. Buhnova. 2021. Using process mining for Git log analysis of projects in a software development course. *Educational and Information Technologies* 26 (2021), 5939–5969.
- [11] M. Mittal and A. Sureka. 2014. Process Mining Software Repositories from Student Projects in an Undergraduate Software Engineering Course. In *Proceedings of the International Conference on Software Engineering (ICSE) Companion*. 344–353.
- [12] S. Onoue, H. Hata, and K. Matsumoto. 2013. A study of the characteristics of developers' activities in GitHub. In *Proceedings of the International Conference on Asia-Pacific Software Engineering Conference*. 7–12.
- [13] W. Poncin, A. Serebrenik, and M. van den Brand. 2011. Process Mining Software Repositories. In *Proceedings of the European Conference on Software Maintenance and Reengineering (CSMR)*. 5–14.
- [14] M. Pons, J.-M. Bruel, J.-B. Raclet, and F. Silvestre. 2023. Finding Behavioral Indicators from Contextualized Commits in Software Engineering Courses with Process Mining. In *Proceedings of the Frontiers in Software Engineering Education (FISEE)*. 56–68.
- [15] V. Rubin, C.W. Günther, W.M.P. van der Aalst, E. Kindler, B.F. van Dongen, and W. Schäfer. 2007. Process Mining Framework for Software Processes. In *Proceedings of the International Conference on Software Process (ICSP)*. 169–181.
- [16] V. Shynkarenko and O. Zhevaho. 2021. Application of Constructive Modeling and Process Mining Approaches to the Study of Source Code Development in Software Engineering Courses. *Journal of Communications Software and Systems* 17, 4 (2021).
- [17] P.-N. Tan, M. Steinbach, and A. Kumar. 2019. *Introduction to Data Mining*. Pearson.
- [18] T. Wambsganss, A. Schmitt, T. Mahnig, A. Ott, S. Soellner, N.A. Ngo, J. Geyer-Klingenberg, J. Nakladal, and J.M. Leimeister. 2021. The Potential of Technology-Mediated Learning Processes: A Taxonomy and Research Agenda for Educational Process Mining. In *Proceedings of the International Conference on Information Systems*. 1–17.
- [19] L. Yan, S. Greiff, J.M. Lodge, and D. Gašević. 2025. Distinguishing performance gains from learning when using generative AI. *Nature Reviews Psychology* 4 (2025), 435–436.
- [20] Y. Yu, G. Yin, H. Wang, and T. Want. 2014. Exploring the patterns of social behavior in GitHub. In *Proceedings of the International Workshop on Crowd-Based Software Development Methods and Technologies*. 31–36.
- [21] M.A. Zayed and A.B. Farid. 2016. The Discovery of the Implemented Software Engineering Process Using Process Mining Techniques. *International Journal of Advanced Computer Science and Applications* 7, 2 (2016).